ÉSTIAM PARIS · 4BLOCKC / E4CCSN · Blockchain Security

Smart Contract Security Audit

Find it · Fix it · Present it — auditing VulnerableVault.sol

Group: Madeira

Individual submission: Zayd ZAYANI

Trainer: David de Paula Santos Silva · Day 4 Presentation

What VulnerableVault.sol Does

An ETH staking vault with a periodic reward lottery

1

Deposit

Users send ETH via deposit(). Their balance is tracked and they are added to the stakers list.

2

Withdraw

Users can pull some or all of their staked ETH back out at any time via withdraw().

3

pickWinner()

Once per day, one staker is pseudo-randomly chosen to receive 100 reward tokens.

It compiles and runs — but it is not safe.

Nine distinct vulnerabilities were planted across access control, reentrancy, randomness, unchecked returns, and the reward business logic. Some are caught by static analysis; the most damaging logic flaws are not.

Methodology: Tool + Human

A tool alone was never going to be enough

S Static Analysis — Slither

Ran on solc 0.8.20 · 102 detectors

Flagged reentrancy, weak randomness, unchecked transfer, missing zero-checks, low-level calls

14 results on the original contract

Fast, but blind to business intent — it cannot tell you a function is missing its owner check, only pattern-match on code shapes.

M Manual Review — Module 11

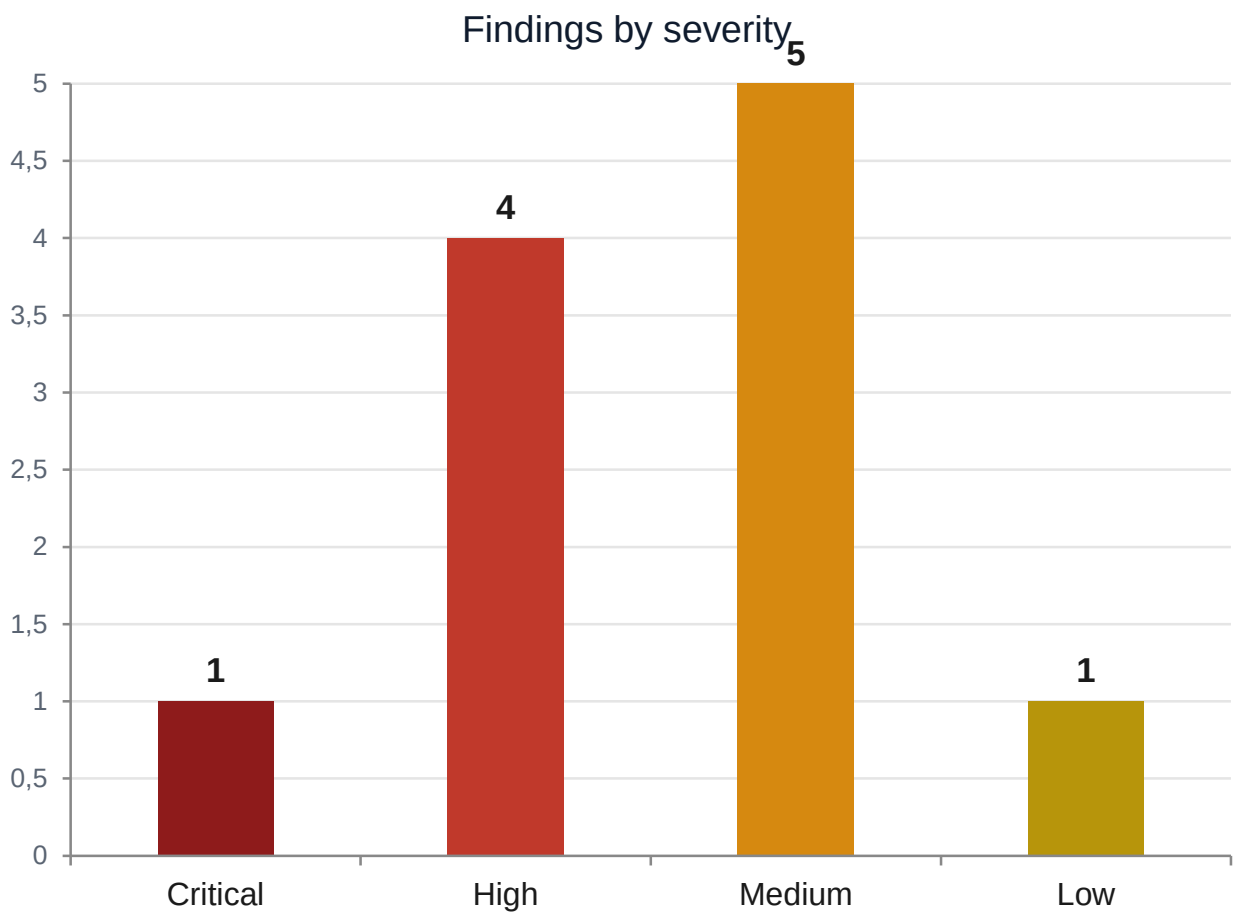
Read every function and asked:

- Who is allowed to call this?
- Can a value be gamed?
- Does the accounting add up?

Found the two highest-value logic bugs in this audit — reward-ticket stuffing and ghost stakers — that Slither never flagged.

Eleven Findings, Ranked by Severity

4 tool-detectable categories · 3 pure business-logic flaws · misc. hygiene



F-01	emergencyWithdraw() — no access control
F-02/03/10	setOwner · reentrancy · permissionless pickWinner (LOGIC)
F-04	Reward-ticket stuffing (LOGIC)
F-05/06/07/11	tx.origin · weak PRNG · unchecked transfer · unchecked arithmetic
F-08	Ghost stakers keep winning odds (LOGIC)
F-09	Zero-address checks, minor hygiene

Top Finding — F-01

CRITICAL

emergencyWithdraw() has no access control

BEFORE — anyone can drain the vault

```
function emergencyWithdraw(
    address payable to
) external {
    uint256 bal = address(this).balance;
    (bool sent,) = to.call{value: bal}("");
    require(sent, "rescue failed");
}
```

AFTER — owner-only, non-zero destination

```
function emergencyWithdraw(
    address payable to
) external onlyOwner nonReentrant {
    require(to != address(0), "zero destination");
    uint256 bal = address(this).balance;
    (bool sent,) = to.call{value: bal}("");
    require(sent, "rescue failed");
}
```

Impact

No modifier restricts the caller. Any address can call emergencyWithdraw(attacker) and redirect 100% of the vault's ETH balance in one transaction — every depositor's stake, gone.

Fix: added the (corrected) onlyOwner modifier and a zero-address check.

The Logic Bug No Tool Could Find — F-04

HIGH

Reward-ticket stuffing via duplicate entries in stakers[]

deposit() — one line, one flaw

```
function deposit() external payable {
  require(msg.value > 0, "zero deposit");
  stakers.push(msg.sender);   // <-- every call!
  balances[msg.sender] += msg.value;
  totalStaked += msg.value;
}
```

Fix — one entry per address

```
if (!isStaker[msg.sender]) {
  isStaker[msg.sender] = true;
  stakers.push(msg.sender);
}
balances[msg.sender] += msg.value;
totalStaked += msg.value;
```

Why Slither can't see this

pickWinner() picks one entry in stakers[] uniformly at random — one entry, one “ticket”. Nothing stops an address from calling deposit() thousands of times with 1 wei each, buying thousands of tickets for almost nothing, while a genuine large staker who deposits once still holds a single ticket. This requires understanding intent, not pattern-matching code.

A Second Logic Bug — F-10

HIGH

pickWinner() is permissionless — enables reward grinding

pickWinner() — anyone can call it

```
function pickWinner() external {
  require(... too soon ...);
  uint256 rand = uint256(keccak256(
    abi.encodePacked(block.timestamp,
    blockhash(block.number-1),
    stakers.length)
  ));
  // pays stakers[rand % stakers.length]
}
```

Fix — restrict who can trigger it

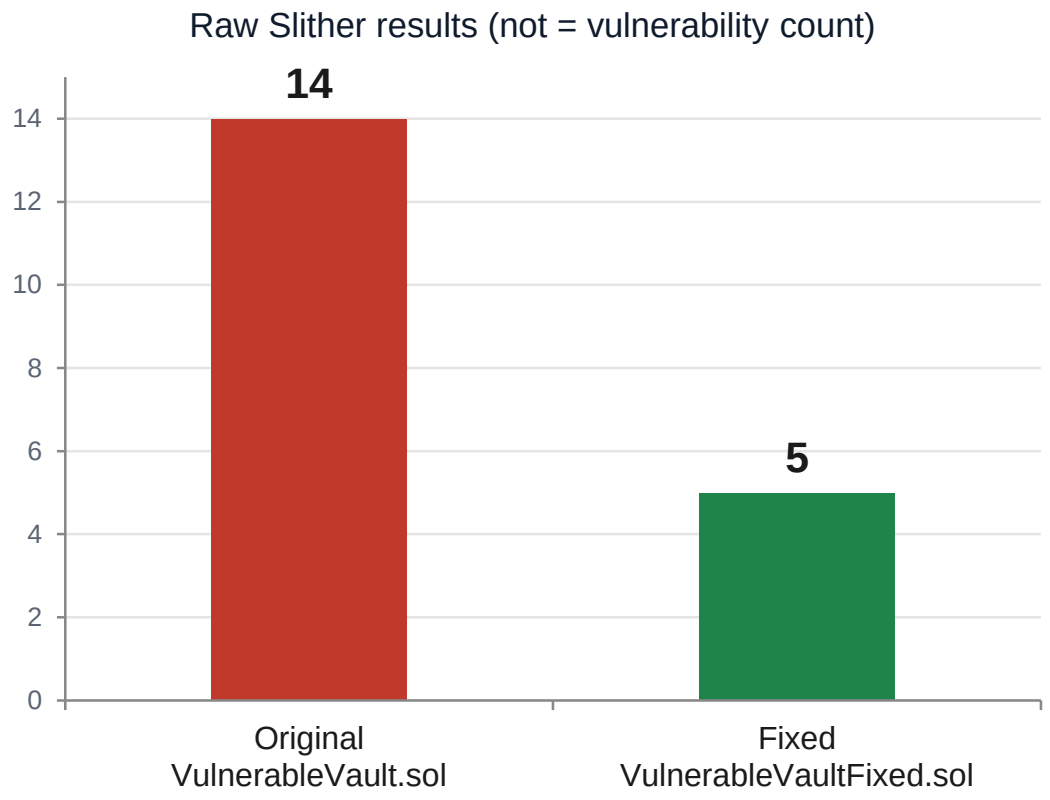
```
function pickWinner()
  external onlyOwner nonReentrant {
  ...
  // same random formula, but only
  // a trusted party decides WHEN
  // to submit the triggering tx
}
```

Why this compounds the weak-randomness finding (F-06)

Weak randomness alone is a modest residual risk. But because pickWinner() has no access control, an attacker can deploy a helper contract that calls it and reverts the whole transaction unless the result favours them — retrying block after block for free until it lands. An unlimited-attempts grinding attack is far more dangerous than a single unpredictable guess.

Proving the Fix Works

Re-running Slither on VulnerableVaultFixed.sol



Note: 14 and 5 are raw analyzer results, not distinct vulnerabilities — some are duplicates (3 reentrancy-tagged results = 1 root cause) or purely informational (solc-version). The 11 numbered findings in the report are the deduplicated count.

Resolved (10 findings)

Reentrancy · both missing access controls · tx.origin · unchecked transfer · unchecked arithmetic · missing zero-checks · all three logic bugs (ticket stuffing, ghost stakers, permissionless pickWinner)

Residual — accepted & documented (1)

• Weak PRNG (F-06) — the grinding path is closed (F-10 fix), but true unpredictability still needs Chainlink VRF for production

Slither's other 4 remaining raw results (timestamp, 2× low-level-calls, solc-version) are accepted, intentional-design, or compiler-level notices — not open findings.

The Most Important Lesson

“A static analysis tool is a triage aid, not an audit.”

Slither instantly caught the reentrancy, the weak randomness, and the unchecked transfer. But it had no way of knowing that setOwner() and emergencyWithdraw() were missing access control entirely, that this reward mechanism was supposed to weight one ticket per staker rather than per transaction, or that leaving pickWinner() open to anyone turns a residual randomness weakness into a practical grinding attack. Those three findings — worth as many marks as every tool-detectable issue combined — only surfaced by reading the specification and asking, function by function: can someone abuse this?

Thank you — questions welcome.